M4 — Demo & Viva Report

MediOps — Smart Hospital Dashboards (Project P7, PRD-08)

Team: Greeshma & Sudhanva Holla Date: 16 July 2026

---

1. What We Built

MediOps is the Phase-1, S-tier slice of PRD-08 (Smart Hospital Dashboards) — the last of the eight Hospital Operations Suite sub-projects, which consumes event data from the other seven and turns it into a live dashboard, an automated statutory report, and a privacy-safe access-control layer.

We implemented five requirements at S-tier level: FR-3 (role-based dashboards with scoped drill-down), FR-5 (HMIS-style monthly report with a review-and-submit workflow), FR-10 (de-identified-by-default views with small-cell suppression), and FR-14 (audit of every dashboard view, drill-down, reveal attempt, and report action). FR-7 (scheduled digests) was scoped in at M1 but not completed — noted honestly below.

The build is a Python stdlib backend (http.server + sqlite3, zero external dependencies) implementing the PRD-08 star-schema data model, with a plain HTML/CSS/JS frontend. The database is rebuilt and deterministically re-seeded on every startup, so the demo is repeatable every single time — the same 6 wards, same event history, same KPI numbers, every run.

Three personas demonstrate the access-control model end-to-end:

- Dr. A. Rao, Medical Superintendent — full access, all wards, can reveal identified data, can generate/submit reports, can view the audit log
- Dr. S. Nair, Department Head (ICU) — scoped to ICU only, can reveal within that scope, cannot generate reports or view the audit log
- P. Menon, Data Protection Officer — sees all wards but can never reveal identified data, and can view the audit log (but not generate reports)

2. Why We Built It This Way

Why role-based scoping is enforced server-side, not in the UI. Every API call (/api/kpis, /api/drilldown, /api/audit) takes a role parameter, and the backend — not the browser — decides what data comes back. A Department Head's ward scope is baked into the role definition in server.py; the frontend never even receives rows outside that scope to begin with. This matters because a permission gate that only lives in the UI can be bypassed by anyone who opens dev tools; a gate enforced at the API layer cannot.

Why de-identification defaults to on, and why the DPO is a hard "no." Most roles *can* request identified data and be granted it if permitted. The DPO persona is different by design: can_reveal is hardcoded False with no override path. This mirrors a real governance principle — a privacy officer's job is to audit access, not to hold clinical-level identified access themselves. We also chose to log *denied* reveal attempts, not just successful ones, because an audit trail that only records success stories isn't useful for accountability.

Why the HMIS report is draft-then-submit, not one click. A statutory submission needs a human to own it. POST /api/report/generate only ever creates a draft; a separate POST /api/report/submit call is required to mark it final. This is a deliberate two-step workflow, not a missing feature.

Why KPIs are computed from raw events, not stored as pre-aggregated numbers. Every KPI (occupancy, ALOS, door-to-doctor, etc.) is derived at request time from fact_events in compute_kpis(). This means the numbers are only ever as good as the underlying event data — which is the honest, correct behavior for a system whose entire premise (per the PRD's own risk register) is "garbage in, garbage dashboards." We did not want to fake confidence with hardcoded tiles.

3. Compliance Notes

| Requirement | Regulatory driver | How it's implemented |
|---|---|---|
| De-identified default + small-cell suppression | DPDP Act 2023 — purpose limitation, data minimisation | Patient tokens (PT-xxxx) shown by default; identified fields (patient_name, patient_uhid) are gated server-side; any breakdown cell with <5 records is suppressed and shown as "—" |
| Access & export audit trail | DPDP access-logging obligation + CERT-In 2022 log retention | Every view, drill-down, reveal attempt (granted or denied), and report action writes a row to audit_log with user, role, action, target, and detail |
| Review-and-submit on statutory reports | HMIS/NHM accountability norms + DPDP | Reports are created as draft and require an explicit second action to reach submitted; a human is always the one who "signs off" |
| Role-based access scoping | DPDP least-privilege posture | Each role has an explicit level (full / unit / de-id-only) enforced in every API handler, not just the UI |

---

**GET endpoints**

| **Endpoint** | **Params** | **Purpose** |
|---|---|---|
| GET /api/roles | — | Returns the 3 personas (Superintendent, Dept Head, DPO) with their permission flags (canReveal, canReport, canAudit) — used to populate the login/role picker |
| GET /api/kpis | role | Computes and returns the 6 live KPI tiles (occupancy, active inpatients, admissions today, discharges today, ALOS, door-to-doctor), ward-level occupancy breakdown, and the case-mix matrix. Scoped to the role's ward if applicable. Logs VIEW_DASHBOARD. |
| GET /api/drilldown | role, metric, | Returns row-level events behind a clicked tile. reveal=1 requests identified data — server decides whether to actually grant it based on role. Logs DRILL_DOWN or REVEAL_ATTEMPT_DENIED. |

| Endpoint | Params | Purpose |
|---|---|---|
| | ward, reveal | |
| GET /api/audit | role | Returns the last 200 audit log rows — but only if the role has canAudit; otherwise returns 403 and logs VIEW_AUDIT_DENIED. |

**POST endpoints**

| Endpoint | Body/Params | Purpose |
|---|---|---|
| POST /api/log | role; body: action, target, detail | Lets the **frontend** log its own actions (e.g. toggling wall mode) into the same audit trail, not just backend-driven events |
| POST /api/report/generate | role | Builds the HMIS-format monthly report from events and inserts it as status='draft'. Blocked (403) unless canReport. Logs GENERATE_REPORT. |
| POST /api/report/submit | body: reportId | Flips a draft report to status='submitted', records who/when. Logs SUBMIT_REPORT. |